

LAPORAN PRAKTIKUM STRUKTUR DATA

“SINGLE LINKED LIST”

Disusun Oleh :

Digo Yuandra

2511533017

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Zahran Azaria

Anvaya



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadiran Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-5 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Single Linked List dalam bagian pembahasan Linked List.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan Single Linked List dalam real Project. Melalui proyek proyek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan Single Linked List pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Digo Yuandra

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	2
2.1 Program NodeSLL_2511533017	2
2.2.1 Kode Program.....	2
2.2.2 Penjelasan	2
2.2 Program TambahSLL_2511533017.....	2
2.3.1 Kode Program.....	2
2.3.2 Penjelasan	6
2.3.3 Output.....	8
2.3 Program PencarianSLL_2511533017	8
2.4.1 Kode Program.....	8
2.4.2 Penjelasan	9
2.4.3 Output.....	10
2.4 Program HapusSLL_2511533017	10
2.5.1 Kode Program.....	10
2.5.2 Penjelasan	14
2.5.3 Output.....	15
BAB III KESIMPULAN	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Single Linked List merupakan salah satu bentuk struktur data dari konsep **Linked List** atau senarai berantai. Dalam struktur ini, setiap data disimpan dalam bagian yang disebut **node**. Setiap node tidak harus berada pada lokasi memori yang berurutan, karena penyimpanannya dilakukan secara dinamis. Pada Single Linked List, setiap node hanya memiliki satu pointer yang digunakan untuk menunjuk node berikutnya. Dengan demikian, hubungan antardata berjalan dalam satu arah, yaitu dari node pertama menuju node selanjutnya sampai node terakhir.

Dalam penerapannya, Single Linked List dapat dibedakan menjadi dua jenis, yaitu **circular** dan **non-circular**. Pada bentuk circular, node terakhir akan menunjuk kembali ke node pertama, sedangkan pada bentuk non-circular, node terakhir akan menunjuk ke nilai `null` sebagai tanda akhir dari list. Karakteristik ini membuat Single Linked List cukup fleksibel untuk digunakan dalam berbagai pengelolaan data yang membutuhkan perubahan jumlah elemen secara dinamis.

Single Linked List sering digunakan karena tidak membutuhkan ruang memori yang harus berurutan seperti array. Struktur ini juga memudahkan proses penambahan dan penghapusan data, terutama ketika operasi dilakukan pada posisi tertentu dalam list. Selain itu, penggunaan memorinya relatif lebih sederhana dibandingkan **Double Linked List**, karena setiap node hanya menyimpan satu pointer. Oleh sebab itu, Single Linked List cocok digunakan ketika data perlu dikelola secara dinamis, tetapi tetap ingin menjaga penggunaan memori agar lebih efisien.

1.2 Tujuan

1. Menjelaskan pemahaman dasar mengenai struktur data **Single Linked List**.
2. Mengetahui cara penggunaan **Single Linked List** dalam pembuatan program.
3. Menerapkan operasi atau method yang terdapat pada **Single Linked List** dalam program sederhana.

1.3 Manfaat

1. Menambah pengalaman praktis dalam menerapkan struktur data **Single Linked List** melalui program.
2. Melatih kemampuan dalam menentukan struktur data yang tepat agar program dapat berjalan lebih efisien, rapi, dan terorganisir.

BAB II

PEMBAHASAN

2.1 Program NodeSLL_2511533017

2.2.1 Kode Program

```
package Pekan5_2511533017;

public class NodeSLL_2511533017 {
    // node bagian data
    int data_3017;
    // Pointer ke node berikutnya
    NodeSLL_2511533017 next_3017;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533017(int data_3017)
    {
        this.data_3017 = data_3017;
        this.next_3017 = null;
    }
}
```

Kode Program 2.1 : program NodeSLL_2511533017

2.2.2 Penjelasan

Program ini merupakan bentuk implementasi **Abstract Data Type (ADT)** yang digunakan untuk merepresentasikan node pada struktur **Single Linked List**. Di dalamnya terdapat deklarasi variabel untuk menyimpan data serta variabel next yang berfungsi sebagai pointer menuju node berikutnya. Selain itu, program ini juga dilengkapi dengan konstruktor yang bertugas memberikan nilai awal pada setiap node saat objek dibuat. Pada proses inisialisasi tersebut, pointer next diberi nilai awal null, yang menunjukkan bahwa node belum terhubung dengan node lain.

2.2 Program TambahSLL_2511533017

2.3.1 Kode Program

```
package Pekan5_2511533017;
```

```

package Pekan5_2511533017;

public class TambahSLL_2511533017 {
    public static NodeSLL_2511533017 insertAtFront_3017
(NodeSLL_2511533017 head_3017, int value_3017) {
        NodeSLL_2511533017 new_node_3017 = new
NodeSLL_2511533017 (value_3017);
        new_node_3017.next_3017 = head_3017;
        return new_node_3017;
    }

    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533017 insertAtEnd_3017
(NodeSLL_2511533017 head_3017, int value_3017) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511533017 newNode_3017 = new
NodeSLL_2511533017 (value_3017);

        // jika list kosong maka node jadi head
        if(head_3017 == null) {
            return newNode_3017;
        }
        // simpan head ke variabel sementara
        NodeSLL_2511533017 last_3017 = head_3017;

        //telusuri ke node akhir
        while (last_3017.next_3017 != null) {
            last_3017 = last_3017.next_3017;
        }

        //ubah pointer
        last_3017.next_3017 = newNode_3017;
        return head_3017;
    }

    static NodeSLL_2511533017 GetNode_3017 (int
data_3017) {
        return new NodeSLL_2511533017 (data_3017);
    }
}

```

```

static NodeSLL_2511533017 insertPos_3017
(NodeSLL_2511533017 headNode_3017, int position_3017, int
value_3017) {
    NodeSLL_2511533017 head_3017 = headNode_3017;
    if (position_3017 < 1)
        System.out.println("Invalid Position");
    if (position_3017 == 1) {
        NodeSLL_2511533017 new_node_3017 = new
NodeSLL_2511533017 (value_3017);
        new_node_3017.next_3017 = head_3017;
        return new_node_3017;
    } else {
        while (position_3017-- != 0) {
            if (position_3017 == 1) {
                NodeSLL_2511533017
newNode_3017 = GetNode_3017 (value_3017);
                newNode_3017.next_3017 =
headNode_3017.next_3017;
                headNode_3017.next_3017 =
newNode_3017;
                break;
            }
            headNode_3017 =
headNode_3017.next_3017;
        }
        if (position_3017 != 1)
            System.out.println("Posisi di luar
jangkau");
        return head_3017;
    }
}

public static void printList_3017 (NodeSLL_2511533017
head_3017) {
    NodeSLL_2511533017 curr_3017 = head_3017;
    while (curr_3017.next_3017 != null) {

        System.out.print(curr_3017.data_3017 + "-->");
        curr_3017 = curr_3017.next_3017;
    }
    if (curr_3017.next_3017 == null) {

        System.out.print(curr_3017.data_3017);
    }
    System.out.println();
}
}

```

```

public static void main (String [] args) {
    //buat linked list 2->3->5->6
    NodeSLL_2511533017 head_3017 = new
NodeSLL_2511533017(2);
    head_3017.next_3017 = new
NodeSLL_2511533017(3);
    head_3017.next_3017.next_3017 = new
NodeSLL_2511533017 (5);
    head_3017.next_3017.next_3017.next_3017 = new
NodeSLL_2511533017(6);

    //cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList_3017(head_3017);
    //tambah node baru di depan
    System.out.print("tambah 1 simpul di depan:
");
    int data_3017 = 1;
    head_3017 = insertAtFront_3017(head_3017,
data_3017);
    //cetak update list
    printList_3017(head_3017);
    //tambahkan node abru di belakang
    System.out.print("tambah 1 simpul di belakang
: ");
    int data2_3017 = 7;
    head_3017 = insertAtEnd_3017 (head_3017,
data2_3017);
    //cetak update list
    printList_3017 (head_3017);

    System.out.print("tambah 1 simpul ke data 4:
");
    int data3_3017 = 4;
    int pos_3017 = 4;
    head_3017 = insertPos_3017
(head_3017,pos_3017,data3_3017);
    // cetak update list
    printList_3017 (head_3017);
}
}

```


2.3.2 Penjelasan

Program ini menerapkan konsep Single Linked List dalam sistem antrian pasien rumah sakit. Setiap data pasien disimpan dalam sebuah node yang berisi informasi nama, penyakit, nomor antrian, serta alamat menuju node berikutnya. Node pertama pada antrian disimpan dalam variabel `head_3017`. Apabila antrian masih kosong, maka data pasien pertama yang dimasukkan akan langsung menjadi node awal. Namun, jika antrian sudah memiliki data, pasien baru akan ditempatkan pada bagian paling belakang sehingga urutan antrian tetap berjalan sesuai prinsip First In First Out atau FIFO.

Pada method `insertAtTail_3017`, program digunakan untuk menambahkan pasien baru ke akhir linked list. Proses ini dimulai dengan menambah nilai `counterAntrian_3017` agar setiap pasien memperoleh nomor antrian secara otomatis. Setelah itu, program membuat node baru berdasarkan nama pasien, jenis penyakit, dan nomor antrian. Jika `head_3017` masih bernilai null, maka node baru tersebut menjadi node pertama. Sebaliknya, jika list sudah berisi data, program akan menelusuri node dari awal sampai menemukan node terakhir, kemudian menghubungkan node terakhir tersebut dengan node baru.

Method `deleteHead_3017` digunakan untuk memanggil pasien yang berada pada posisi paling depan. Sebelum proses dilakukan, program akan mengecek apakah antrian kosong atau tidak. Jika tidak ada pasien dalam antrian, maka program menampilkan pesan bahwa antrian masih kosong. Jika terdapat pasien, data pada node pertama akan ditampilkan sebagai pasien yang dipanggil. Setelah itu, posisi `head_3017` dipindahkan ke node berikutnya sehingga pasien yang sudah dipanggil tidak lagi berada dalam daftar antrian.

Selanjutnya, method `printList_3017` berfungsi untuk menampilkan seluruh data pasien yang masih menunggu dalam antrian. Program akan memulai penelusuran dari `head_3017`, lalu bergerak ke node berikutnya sampai mencapai node terakhir. Setiap node akan ditampilkan dengan informasi nomor antrian, nama pasien, dan penyakit. Apabila tidak ada data di dalam list, program akan memberikan keterangan bahwa antrian belum memiliki pasien.

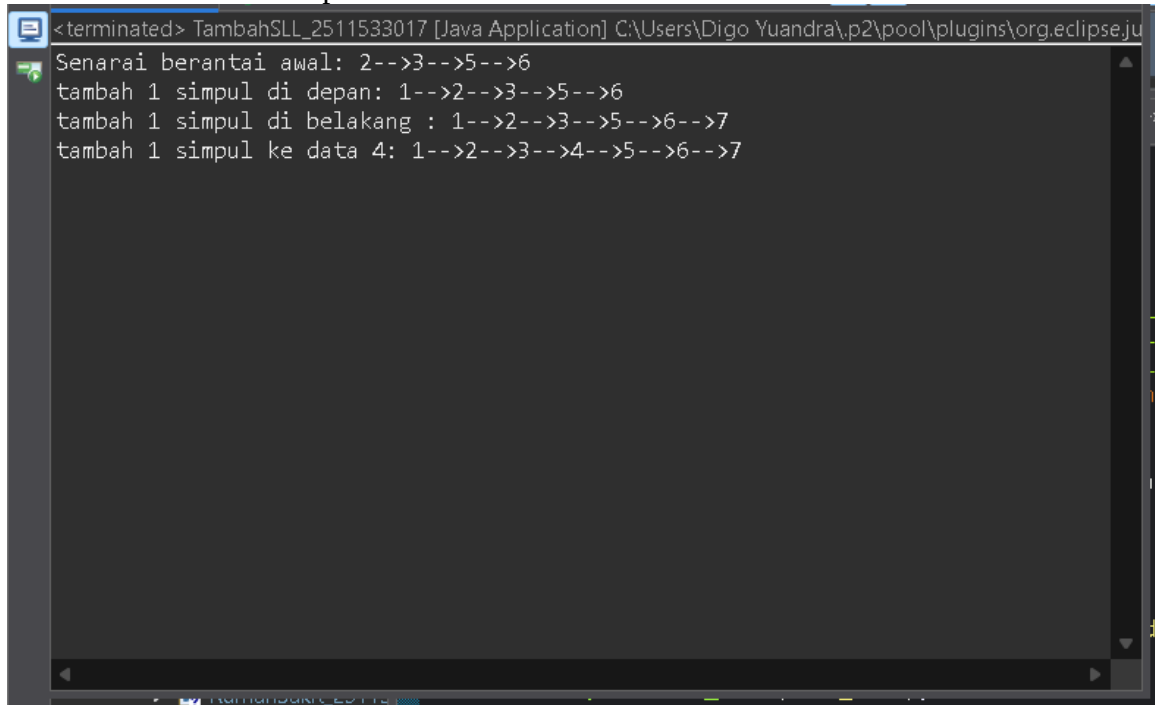
Method `searchPatient_3017` digunakan untuk mencari data pasien berdasarkan nama. Program membandingkan nama yang dimasukkan pengguna dengan nama pasien yang tersimpan pada setiap node. Pencarian ini menggunakan `equalsIgnoreCase`, sehingga huruf besar dan kecil tidak memengaruhi hasil pencarian. Jika nama ditemukan, program akan menampilkan informasi lengkap pasien tersebut. Namun, jika nama tidak ada dalam list, program akan menampilkan pesan bahwa pasien tidak ditemukan.

Method `checkStatus_3017` digunakan untuk mengetahui kondisi antrian secara keseluruhan. Program menghitung jumlah pasien dengan cara menelusuri setiap node yang ada di dalam linked list. Setelah proses perhitungan selesai, program menampilkan total pasien yang sedang menunggu serta informasi pasien yang berada di

urutan paling depan. Dengan method ini, pengguna dapat melihat status antrian tanpa harus membaca seluruh daftar pasien satu per satu.

Pada bagian main, program menyediakan menu utama yang dapat dipilih oleh pengguna. Menu tersebut meliputi pendaftaran pasien, pemanggilan pasien, penampilan daftar antrian, pencarian pasien, pengecekan status antrian, dan keluar dari program. Setiap menu terhubung dengan method tertentu, sehingga alur program menjadi lebih terstruktur dan mudah dipahami.

2.3.3 Output



```
<terminated> TambahSLL_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\pool\plugins\org.eclipse.ju
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang : 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2.1 : Ouput dari program
TambahSLL_2511533017

2.3 Program PencarianSLL_2511533017

2.4.1 Kode Program

```

package Pekan5_2511533017;
public class PencarianSLL_2511533017 {
    static boolean searchKey_3017 (NodeSLL_2511533017
head_3017, int key_3017) {
        NodeSLL_2511533017 curr_3017 = head_3017;
        while (curr_3017 != null) {
            if (curr_3017.data_3017 != key_3017)
                return true;
            curr_3017 = curr_3017.next_3017;
        }
        return false;
    }
    public static void traversal_3017 (NodeSLL_2511533017
head_3017) {
        //mulai dari head
        NodeSLL_2511533017 curr_3017 = head_3017;
        //telusuri sampai pointer null
        while (curr_3017 != null) {
            System.out.print(" " +
curr_3017.data_3017);
            curr_3017 = curr_3017.next_3017;
        }
        System.out.println();
    }
}

```

```

public static void main (String [] args) {
    NodeSLL_2511533017 head_3017 = new
NodeSLL_2511533017 (14);
    head_3017.next_3017 = new NodeSLL_2511533017
(21);
    head_3017.next_3017.next_3017 = new
NodeSLL_2511533017 (13);
    head_3017.next_3017.next_3017.next_3017 = new
NodeSLL_2511533017 (30);

    head_3017.next_3017.next_3017.next_3017.next_3017 =
new NodeSLL_2511533017 (10);
    System.out.print("Penulisan SLL :");
    traversal_3017(head_3017);
    //data yang akan dicari
    int key_3017 = 30;
    System.out.print("cari data " + key_3017 + " =
");
    if (searchKey_3017(head_3017, key_3017))
        System.out.println("Ketemu");
    else
        System.out.println("tidak ada");
    }
}

```

Kode Program 2.3 : Pogram PencarianSLL

2.4.2 Penjelasan

Program ini diawali dengan pembuatan method bertipe boolean bernama searchKey. Method ini digunakan untuk mencari apakah suatu nilai tertentu terdapat di dalam linked list atau tidak. Proses pencarian dilakukan dengan menelusuri node dari bagian head. Selama curr belum bernilai null, program akan memeriksa data yang ada pada node tersebut. Jika data pada curr sama dengan key yang dicari, maka method akan mengembalikan nilai true. Namun, jika data belum sesuai, curr akan dipindahkan ke node berikutnya melalui pointer next. Jika seluruh node sudah diperiksa dan data tidak ditemukan, maka method akan mengembalikan nilai false.

Selanjutnya, terdapat method traversal yang berfungsi untuk menampilkan seluruh data yang tersimpan dalam linked list. Pada method ini, program menggunakan perulangan while untuk menelusuri setiap node dari awal sampai akhir. Selama curr masih berisi data atau belum bernilai null, program akan mencetak nilai yang tersimpan pada node tersebut. Setelah data ditampilkan, curr akan diarahkan ke node berikutnya menggunakan pointer next. Proses ini terus berjalan sampai semua data dalam list selesai ditampilkan.

Pada bagian main, program dimulai dengan memasukkan beberapa data node ke dalam linked list. Setelah data tersimpan, program akan memanggil method traversal untuk menampilkan isi list secara keseluruhan. Kemudian, program melakukan pencarian terhadap nilai tertentu, misalnya key 30, dengan memanggil method searchKey. Jika nilai tersebut ditemukan dalam list, maka program

akan menampilkan pesan bahwa data berhasil ditemukan. Sebaliknya, jika nilai tidak ada dalam list, maka program akan menampilkan pesan bahwa data tidak ditemukan.

2.4.3 Output

```
<terminated> PencarianSLL_2511533017 [Java Application]
Penelusuran SLL : 14 21 13 30 10
cari data 30 = Ketemu
```

2.4.4

Gambar 2.2 : Ouput program PencarianSLL_2511533017

2.4 Program HapusSLL_2511533017

2.5.1 Kode Program

```
package Pekan5_2511533017;
public class HapusSLL_2511533017 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511533017 deleteHead_3017
(NodeSLL_2511533017 head_3017) {
    //jika SLL Kosong
```

```

//pindahkan ke node berikutnya
    head_3017 = head_3017.next_3017;
    //return head baru
    return head_3017;
}

// fungsi menghapus node terakhir SLL
public static NodeSLL_2511533017 removeLastNode_3017
(NodeSLL_2511533017 head_3017) {
    //jika list kosong, return null
    if (head_3017 == null) {
        return null;
    }
    //jika list satu node, hapus node akan return null
    if (head_3017.next_3017 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511533017 secondLast_3017 =
head_3017;
    while (secondLast_3017.next_3017.next_3017 !=
null) {
        secondLast_3017 =
secondLast_3017.next_3017;
    }
    //hapus node terakhir
    secondLast_3017.next_3017 = null;
    return head_3017;
}

public static NodeSLL_2511533017 deleteNode_3017
(NodeSLL_2511533017 head_3017, int position_3017) {
    NodeSLL_2511533017 temp_3017 = head_3017;
    NodeSLL_2511533017 prev_3017 = null;
    //jika linked list null
    if (temp_3017 == null)
        return head_3017;
    //kasus 1 : head dihapus
    if (position_3017 == 1) {
        head_3017 = temp_3017.next_3017;
        return head_3017;
    }
}

```

```

//kasus 2 : menghapus node di tengah
//telusuri ke node yang dihapus

        for (int i = 1; temp_3017 != null && i <
position_3017; i++) {
            prev_3017 = temp_3017;
            temp_3017 = temp_3017.next_3017;
        }
        //jika ditemukan, hapus node
        if (temp_3017 != null) {
            prev_3017.next_3017 =
temp_3017.next_3017;
        }else {
            System.out.println("Data tidak ada");
        }
        return head_3017;
    }
}
//fungsi mencetak SLL
public static void printList_3017 (NodeSLL_2511533017
head_3017) {
    NodeSLL_2511533017 curr_3017 = head_3017;
    while (curr_3017.next_3017 != null) {
        System.out.print(curr_3017.data_3017+"-
->");
        curr_3017 = curr_3017.next_3017;
    }
    if (curr_3017.next_3017 == null) {
        System.out.print(curr_3017.data_3017);
    }
    System.out.println();
}
}

```

```

public static void main (String [] args) {
    //buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511533017 head_3017 = new
NodeSLL_2511533017 (1);
    head_3017.next_3017 = new NodeSLL_2511533017
(2);
    head_3017.next_3017.next_3017 = new
NodeSLL_2511533017 (3);
    head_3017.next_3017.next_3017.next_3017 = new
NodeSLL_2511533017 (4);

    head_3017.next_3017.next_3017.next_3017.next_3017 =
new NodeSLL_2511533017 (5);

    head_3017.next_3017.next_3017.next_3017.next_3017.next_3017= new NodeSLL_2511533017 (6);

    //cetak list awal
    System.out.println("list awal : ");
    printList_3017(head_3017);

    //hapus head
    head_3017 = deleteHead_3017 (head_3017);
    System.out.println("List setelah head di hapus
: ");

    printList_3017(head_3017);
}

```



```

//hapus node terakhir
head_3017 = removeLastNode_3017 (head_3017);
System.out.println("List setelah simpul
terakhir di hapus : ");
printList_3017(head_3017);

//deleting node position 2
int position_3017 = 2;
head_3017 =
deleteNode_3017(head_3017,position_3017);

//print list after deletion
System.out.println("List setelah posisi 2
dihapus: ");
printList_3017(head_3017);

```

Kode Program 2.4 : Program HapusSLL_2511533017

2.5.2 Penjelasan

Program ini diawali dengan penggunaan struktur NodeSLL sebagai dasar pembentukan node pada Single Linked List. Pada tahap awal, program melakukan pengecekan terhadap kondisi head. Jika head masih bernilai null, berarti list belum memiliki data sehingga program akan mengembalikan nilai null. Pemeriksaan ini dilakukan agar program tidak menjalankan proses penghapusan pada list yang kosong.

Method `removeLastNode` digunakan untuk menghapus node yang berada di bagian paling akhir dari list. Prosesnya dimulai dengan memeriksa apakah head bernilai null atau tidak. Jika list kosong, maka tidak ada node yang dapat dihapus. Selanjutnya, program juga memeriksa apakah list hanya memiliki satu node. Jika hanya terdapat satu node, maka node tersebut akan dihapus dengan cara mengembalikan nilai null, sehingga list menjadi kosong.

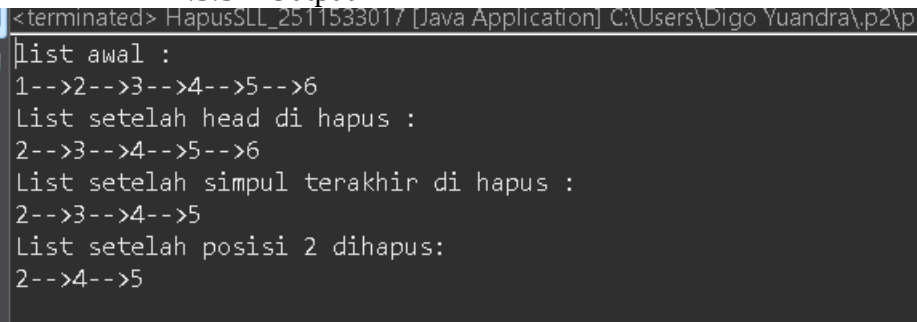
Apabila list memiliki lebih dari satu node, maka program akan mencari node sebelum node terakhir. Pencarian ini dilakukan dengan menggunakan variabel sementara bernama `secondLast` yang awalnya menyimpan nilai head. Program kemudian menjalankan perulangan `while` untuk menelusuri list sampai menemukan node kedua terakhir. Selama node setelah node berikutnya dari `secondLast` belum bernilai null, maka posisi `secondLast` akan terus berpindah ke node berikutnya. Setelah node kedua terakhir ditemukan, pointer `next` dari `secondLast` akan diubah menjadi null. Dengan cara ini, node terakhir tidak lagi terhubung ke dalam list dan dianggap telah terhapus.

Selain itu, program juga memiliki method `deleteNode` yang digunakan untuk menghapus node berdasarkan nilai tertentu. Method ini akan menelusuri isi list untuk menemukan node yang memiliki data sesuai dengan nilai yang ingin dihapus. Jika node tersebut ditemukan, maka hubungan pointer akan diatur ulang agar node tersebut dilewati oleh list. Dengan demikian, node yang memiliki nilai tertentu dapat dihapus tanpa harus menghapus seluruh isi list.

Program ini juga dilengkapi dengan method `printList` yang berfungsi untuk menampilkan seluruh data node yang masih tersimpan dalam list. Method ini bekerja dengan menelusuri node dari head sampai node terakhir. Setiap nilai yang ada di dalam node akan ditampilkan secara berurutan, sehingga pengguna dapat melihat kondisi list sebelum maupun sesudah proses penghapusan dilakukan.

Pada bagian main, program diawali dengan memasukkan beberapa nilai node ke dalam linked list. Setelah data berhasil ditambahkan, program menampilkan isi list dengan memanggil method `printList`. Kemudian, program melakukan beberapa proses penghapusan, yaitu menghapus node pertama dengan memanggil method `deleteHead`, menghapus node terakhir dengan method `removeLastNode`, serta menghapus node pada posisi tertentu menggunakan method `deleteNode`. Setelah seluruh proses penghapusan selesai, program kembali menampilkan isi list untuk memperlihatkan perubahan data setelah node awal, node akhir, dan node tertentu dihapus.

2.5.3 Output



```
<terminated> HapusSLL_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\p
list awal :
1-->2-->3-->4-->5-->6
List setelah head di hapus :
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus :
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.3 : output program HapusSLL

BAB III

KESIMPULAN

Single Linked List merupakan salah satu struktur data dinamis yang tersusun dari beberapa node yang saling terhubung secara satu arah. Setiap node memiliki dua bagian penting, yaitu data yang disimpan dan pointer next yang mengarah ke node berikutnya. Berbeda dengan array yang biasanya memiliki ukuran tetap dan dapat diakses langsung melalui indeks, Single Linked List menggunakan alokasi memori secara dinamis sesuai jumlah data yang dimasukkan. Untuk mengakses datanya, program harus menelusuri node secara berurutan mulai dari head sampai mencapai node terakhir yang menunjuk ke null. Oleh karena itu, penggunaan pointer harus dilakukan dengan hati-hati, karena kesalahan dalam mengatur hubungan antar-node dapat menyebabkan data terputus atau tidak dapat diakses kembali.

Kelebihan Single Linked List terdapat pada kemampuannya dalam mengelola memori secara lebih fleksibel. Struktur ini juga memudahkan proses penambahan dan penghapusan data, terutama pada bagian tertentu dari list, tanpa harus menggeser banyak elemen seperti pada array. Namun, Single Linked List juga memiliki kekurangan, yaitu tidak dapat mengakses data secara langsung berdasarkan indeks dan membutuhkan tambahan memori untuk menyimpan pointer. Meskipun demikian, struktur data ini sangat penting untuk dipahami karena menjadi dasar dalam mempelajari alokasi memori dinamis, hubungan antar-node, dan pengelolaan referensi dalam pemrograman. Pemahaman terhadap Single Linked List juga membantu melatih ketelitian dalam menangani kondisi list kosong, node pertama, node terakhir, serta perubahan pointer saat proses penyisipan dan penghapusan data.

DAFTAR PUSTAKA

- [1] "Struktur Data: Linked List," *Trivusi*, Jul. 2022. [Online]. Available: <https://www.trivusi.web.id/2022/07/struktur-data-linked-list.html>. [Accessed: May 5, 2026].
- [2] Candra, "Makalah Single Linked List dalam Bahasa Pemrograman C," *SlideShare*, 2023. [Online]. Available: <https://www.slideshare.net/slideshow/makalah-single-linked-list-dalam-bahasadocx/254436610>. [Accessed: May 5, 2026].

LAPORAN TUGAS

A. Program Pasien_2511533017

```
package Pekan5_2511533017; // Deklarasi package untuk mengorganisir kelas

public class Pasien_2511533017 { // Kelas ADT yang merepresentasikan node
    dalam Single Linked List
    // Deklarasi atribut data pasien dan pointer next dengan suffix NIM
    String nama_3017;
    String penyakit_3017;
    int no_3017;
    Pasien_2511533017 next_3017;

    // Constructor untuk menginisialisasi atribut saat objek node dibuat
    public Pasien_2511533017 (String nama_3017, String penyakit_3017, int
no_3017) {
        this.nama_3017 = nama_3017;
        this.penakit_3017 = penyakit_3017;
        this.no_3017 = no_3017;
        this.next_3017 = null;
    }

    // Method getter untuk mengakses nilai atribut privat
    public String getNama_3017() {return nama_3017;}
    public String getPenyakit_3017() {return penyakit_3017;}
    public int getNomor_3017() { return no_3017;}
    public Pasien_2511533017 getNext() {return next_3017;}

    // Method setter untuk mengubah nilai atribut dan menghubungkan pointer
next
    public void setNama_3017 (String nama_3017) {this.nama_3017 = nama_3017;}
    public void setPenyakit_3017 (String penyakit_3017) {this.penakit_3017 =
penyakit_3017;}
    public void setNomor_3017 (int no_3017) {this.no_3017 = no_3017;}
    public void setNext_3017 (Pasien_2511533017 next_3017) {this.next_3017 =
next_3017;}
}
```

B. Program RumahSakit_2511533017

```
package Pekan5_2511533017;

import java.util.Scanner;

public class RumahSakit_2511533017 {

    // Counter global untuk nomor antrian auto-increment
    static int counterAntrian_3017 = 0;
```

```
// Insert at Tail: menambahkan node pasien baru di akhir linked list dengan
prinsip FIFO
public static Pasien_2511533017 insertAtTail_3017(Pasien_2511533017
head_3017, String nama_3017, String penyakit_3017) {
    counterAntrian_3017++;
    Pasien_2511533017 newNode_3017 = new Pasien_2511533017(nama_3017,
penyakit_3017, counterAntrian_3017);

    if (head_3017 == null) {
        return newNode_3017;
    }

    Pasien_2511533017 last_3017 = head_3017;
    while (last_3017.getNext() != null) {
        last_3017 = last_3017.getNext();
    }

    last_3017.setNext_3017(newNode_3017);
    return head_3017;
}

// Delete Head: menghapus dan menampilkan data pasien terdepan lalu
menggeser head ke node berikutnya
public static Pasien_2511533017 deleteHead_3017(Pasien_2511533017
head_3017) {
    if (head_3017 == null) {
        System.out.println("Antrian kosong! Tidak ada pasien untuk
dipanggil");
        return null;
    }

    Pasien_2511533017 pasienDipanggil_3017 = head_3017;
    System.out.println("\n=== Pasien Dipanggil ===");
    System.out.println("Nama      : " +
pasienDipanggil_3017.getNama_3017());
    System.out.println("Penyakit   : " +
pasienDipanggil_3017.getPenyakit_3017());
    System.out.println("Nomor Antrian: " +
pasienDipanggil_3017.getNomor_3017());

    return head_3017.getNext();
}

// Display List: menelusuri linked list dari head hingga null dan
menampilkan semua data pasien
public static void printList_3017(Pasien_2511533017 head_3017) {
    if (head_3017 == null) {
        System.out.println("Antrian kosong. Belum ada pasien terdaftar");
        return;
    }

    System.out.println("\n=== Daftar Antrian ===");
    Pasien_2511533017 curr_3017 = head_3017;
```

```

        while (curr_3017.getNext() != null) {
            System.out.println("[ " + curr_3017.getNomor_3017() +
                curr_3017.getNama_3017() + " - " +
                curr_3017.getPenyakit_3017() + " -
->");
            curr_3017 = curr_3017.getNext();
        }

        if (curr_3017 != null) {
            System.out.println("[ " + curr_3017.getNomor_3017() +
                curr_3017.getNama_3017() + " - " +
                curr_3017.getPenyakit_3017());
        }
    }

    // Search Case-Insensitive: mencari pasien berdasarkan nama
    tanpa membedakan huruf besar atau kecil
    public static void searchPatient_3017(Pasien_2511533017
head_3017, String namaCari_3017) {
        if (head_3017 == null) {
            System.out.println("Antrian kosong");
            return;
        }

        Pasien_2511533017 curr_3017 = head_3017;
        boolean ditemukan_3017 = false;
        while (curr_3017 != null) {
            if
(curr_3017.getNama_3017().equalsIgnoreCase(namaCari_3017)) {
                System.out.println("\n=== Pasien Ditemukan ===");
                System.out.println("Nama          : " +
curr_3017.getNama_3017());
                System.out.println("Penyakit       : " +
curr_3017.getPenyakit_3017());
                System.out.println("Nomor Antrian: " +
curr_3017.getNomor_3017());
                ditemukan_3017 = true;
                break;
            }
            curr_3017 = curr_3017.getNext();
        }

        if (!ditemukan_3017) {
            System.out.println("Pasien dengan nama '" +
namaCari_3017 + "' tidak ditemukan");
        }
    }

```

```

// Cek Status Antrian: menghitung total pasien dan menampilkan informasi
pasien terdepan
public static void checkStatus_3017(Pasien_2511533017 head_3017) {
    if (head_3017 == null) {
        System.out.println("Status Antrian: KOSONG. Tidak ada pasien");
        return;
    }

    int total_3017 = 0;
    Pasien_2511533017 curr_3017 = head_3017;

    while (curr_3017 != null) {
        total_3017++;
        curr_3017 = curr_3017.getNext();
    }

    System.out.println("\n=== Status Antrian ===");
    System.out.println("Total Pasien      : " + total_3017);
    System.out.println("Pasien Terdepan : " + head_3017.getNama_3017() +
        " (No. " + head_3017.getNomor_3017() + ")");
}

// Menu utama: menangani input pengguna dan memanggil method operasi
linked list yang sesuai
public static void main(String[] args) {
    Scanner input_3017 = new Scanner(System.in);
    Pasien_2511533017 head_3017 = null;
    int pilihan_3017 = 0;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533017
===");

        System.out.println("1. Daftarkan Pasien (Insert at Tail)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
    } while (true);
}

```



```

        if (input_3017.hasNextInt()) {
            pilihan_3017 = input_3017.nextInt();
            input_3017.nextLine();
        } else {
            System.out.println("Input tidak valid. Silakan
masukkan angka 1-6");
            input_3017.nextLine();
            continue;
        }

        switch (pilihan_3017) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_3017 = input_3017.nextLine();

                System.out.print("Masukkan Penyakit   : ");
                String penyakit_3017 = input_3017.nextLine();

                head_3017 = insertAtTail_3017(head_3017,
nama_3017, penyakit_3017);
                System.out.println("Pasien berhasil
didaftarkan! Nomor Antrian: " + counterAntrian_3017);
                break;

            case 2:
                head_3017 = deleteHead_3017(head_3017);
                break;

            case 3:
                printList_3017(head_3017);
                break;

            case 4:
                System.out.print("Masukkan Nama Pasien yang
dicari: ");
                String cari_3017 = input_3017.nextLine();
                searchPatient_3017(head_3017, cari_3017);
                break;

            case 5:
                checkStatus_3017(head_3017);
                break;

            case 6:
                System.out.println("Program selesai. Terima
kasih");
                break;

            default:
                System.out.println("Pilihan tidak valid.
Silakan pilih 1-6");
        }

    } while (pilihan_3017 != 6);

    input_3017.close();
}
}

```

*penjelasan program adalah berupa baris komen pada program.

OUTPUT PROGRAM :

```
RumahSakit_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe
=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: digo
Masukkan Penyakit : demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan:
```

RumahSakit_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\pool\

```

=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: siddig
Masukkan Penyakit : batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2

```

```

=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

```

```

=== Daftar Antrian ===
[1] digo - demam -->
[2] siddig - batuk

```

RumahSakit_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\pool\

```

1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

```

```

=== Pasien Dipanggil ===
Nama      : digo
Penyakit   : demam
Nomor Antrian: 1

```

```

RumahSakit_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\pool\p
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: siddiq
Pasien dengan nama 'siddiq' tidak ditemukan

=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: siddig

=== Pasien Ditemukan ===
Nama      : siddig
Penyakit  : batuk
Nomor Antrian: 2

RumahSakit_2511533017 [Java Application] C:\Users\Digo Yuandra\p2\p
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5

=== Status Antrian ===
Total Pasien      : 1
Pasien Terdepan   : siddig (No. 2)

=== Antrian Rumah Sakit NIM: 2511533017 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai. Terima kasih

```